

Project Proposal & Mobile Architecture Plan

AI-Based Vision Assistance System

Implementation Strategy: Android App via Capacitor

Prepared By:

Om Dombe

System Architect & Lead Developer

Date:

February 15, 2026

Abstract

This document outlines the architectural strategy for developing a mobile-based "Vision-to-Audio" assistance application. By leveraging **Capacitor** and **React**, we aim to deploy a cross-platform solution that utilizes client-side AI (TensorFlow.js) for offline object detection and native text-to-speech plugins. This approach minimizes hardware costs by utilizing the user's existing Android smartphone while ensuring high portability and real-time performance.

Contents

1	Introduction and Objective	2
1.1	Problem Statement	2
1.2	Proposed Solution (Hybrid Mobile App)	2
2	System Architecture	2
2.1	Architectural Diagram	2
2.2	Tech Stack	2
3	Implementation Strategy	3
3.1	Module 1: Environmental Setup	3
3.2	Module 2: The Camera Layer	3
3.3	Module 3: AI Inference (Client-Side)	3
3.4	Module 4: Native Audio Feedback	4
4	Required Design Documentation	4
4.1	1. Mobile UX/Accessibility Design	4
4.2	2. Permission Flow Diagram	4
4.3	3. Battery Optimization Plan	4
5	Resource Analysis	4
5.1	Advantages over Hardware Prototype	4
6	Conclusion	5

1 Introduction and Objective

1.1 Problem Statement

Dedicated hardware solutions (like Raspberry Pi wearables) can be expensive and cumbersome for users to carry. A smartphone-based solution offers higher processing power, better battery life, and superior portability without additional hardware costs.

1.2 Proposed Solution (Hybrid Mobile App)

We propose a **Hybrid Mobile Application** built using **React** and wrapped with **Capacitor** to run natively on Android.

- **Framework:** Ionic Capacitor (Bridging Web Code to Native Android).
- **AI Engine:** TensorFlow.js (Running optimized models in the WebView).
- **Output:** Native Text-to-Speech (TTS) via Capacitor Plugins.
- **Accessibility:** "Blind-first" UI design with gesture-based controls.

2 System Architecture

The system architecture differs from the laptop version by introducing a "Bridge" layer that connects the JavaScript logic to the phone's hardware.

2.1 Architectural Diagram

1. **Native Layer (Android):** Manages Camera Hardware, Speakers, and Battery.
2. **Capacitor Bridge:** Facilitates communication between the Native Layer and the WebView.
3. **Web Layer (React App):**
 - **View:** A full-screen camera preview (using 'capacitor-camera-preview').
 - **Logic:** 'requestAnimationFrame' loop capturing frame data.
 - **Inference:** TensorFlow.js processing frames via WebGL backend.

2.2 Tech Stack

- **Frontend Framework:** React.js (Vite template).
- **Native Runtime:** Capacitor 6.0.
- **Object Detection:** TensorFlow.js ('coco-ssd' or converted YOLO model).

- **OCR (Text Reading):** Tesseract.js (Offline Version).
- **Speech:** '@capacitor-community/text-to-speech'.

3 Implementation Strategy

3.1 Module 1: Environmental Setup

Objective: Configure the Hybrid Development Environment.

1. Initialize React App: `npm create vite@latest glauc-app -- --template react`.
2. Install Capacitor: `npm install @capacitor/core @capacitor/cli @capacitor/android`.
3. Initialize Capacitor: `npx cap init`.
4. Add Android Platform: `npx cap add android`.

3.2 Module 2: The Camera Layer

Objective: High-performance video stream access.

- Standard HTML5 '<video>' elements can be slow in WebViews.
- **Solution:** Use '@capacitor-community/camera-preview'.
- This plugin renders the native camera stream *behind* the WebView (HTML), allowing for zero-latency preview while the HTML layer sits on top for UI and AI processing.

3.3 Module 3: AI Inference (Client-Side)

Objective: Offline object detection without API calls.

- Install TFJS: `npm install @tensorflow/tfjs @tensorflow-models/coco-ssd`.
- **Backend Selection:** Force the 'webgl' backend for GPU acceleration on the phone.
- **Implementation:**

```
// Pseudo-code for detection loop
const detect = async (video) => {
  const predictions = await model.detect(video);
  if (predictions.score > 0.7) {
    handleAlert(predictions.class);
  }
  requestAnimationFrame(() => detect(video));
};
```

3.4 Module 4: Native Audio Feedback

Objective: Clear, loud voice alerts.

- Web Speech API is unreliable in background modes.
- **Solution:** Use Capacitor TTS Plugin.
- **Logic:** Maintain a "Debounce" timer to prevent the app from repeating "Chair... Chair... Chair" 10 times a second.

4 Required Design Documentation

4.1 1. Mobile UX/Accessibility Design

- Since the user is visually impaired, the screen UI is secondary.
- Define ****Gesture Controls****:
 - Single Tap: "What is in front of me?" (Scene Description).
 - Double Tap: "Read text" (OCR Mode).
 - Shake: "Stop/Reset".

4.2 2. Permission Flow Diagram

- Android 13+ requires strict runtime permissions.
- Document the flow for requesting **CAMERA** and **AUDIO** permissions on the first launch.

4.3 3. Battery Optimization Plan

- Running AI models drains battery fast.
- Strategy document: "Throttle AI to 5 frames per second (FPS) to preserve battery life."

5 Resource Analysis

5.1 Advantages over Hardware Prototype

Feature	Raspberry Pi Version	Capacitor Android App
Cost	\$150+ (Hardware)	\$0 (Uses User Phone)
Power	External Battery Pack	Phone Battery
Deployment	Complex Hardware Setup	Simple APK Install
Performance	Limited (Pi CPU)	High (Snapdragon/Pixel chips)

6 Conclusion

The Capacitor-based approach transforms the project from a hardware engineering challenge into a modern software development lifecycle. By utilizing `TensorFlow.js` and `React`, we leverage the immense computing power of modern smartphones to deliver a cost-effective, real-time solution for visually impaired users.